K-ERP Stock & Purchase Module

Technical Report

Requirements, architecture, design, tests and quality evaluation

Item	Value
Document ID	KERP-STK-TR-001
Version	1.0
Date	21 October 2026
Author	Kongkat Thanalertrungroj
System	K-ERP Stock & Purchase (Stock-Purchase-Backend + Stock-Purchase-Frontend)
Live demo	https://k-erp-hub.vercel.app
Source	github.com/kingly5556/Stock-Purchase-Backend · Stock-Purchase-Frontend · K-ERP
Companion document	KERP-ACC-TR-001 (Accounting / General Ledger technical report)

Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Intended audience 3
1.4 Definitions and acronyms 3
1.5 Normative references 3
1.6 Document overview 3
2. System overview 5
2.1 Background 5
2.2 Product perspective 5
2.3 Product functions 5
2.4 User classes 5
2.5 Operating environment 6
2.6 Design constraints 6
2.7 Assumptions and dependencies 6
3. Software requirements specification 7
3.1 Functional requirements 7
3.2 Business rules 7
3.3 Quality requirements (per ISO/IEC 25010) 8
3.4 External interface requirements 8
4. Architecture description 9
4.1 Stakeholders and concerns 9
4.2 Viewpoints 9
4.3 Context view 9
4.4 Logical (module) view 9
4.5 Information view 9
4.6 Interaction view: transactional outbox 10
4.7 Lifecycle view: purchase order 10
4.8 Deployment view 11
4.9 Security view 11
4.10 Architecture decision records 11
5. Detailed design 13
5.1 Posting a stock movement 13
5.2 Receiving goods 13
5.3 Purchase order numbering and status changes 13
5.4 Accounting events 13
5.5 Error model 13
5.6 Configuration 14
6. Test documentation 15
6.1 Test plan 15
6.2 Test design 15
6.3 Test completion report 16
6.4 Incident (anomaly) report 17
7. Quality evaluation (ISO/IEC 25010) 18
7.1 Findings and recommendations 18
8. Requirements traceability matrix 19
9. Limitations, risks and future work 20
9.1 Intentional limitations 20
9.2 Risks 20
9.3 Future work 20
10. Conclusion 21
Appendix A. API reference 22
Appendix B. Data dictionary 23

1. Introduction

1.1 Purpose

This report describes the Stock & Purchase module of K-ERP: what it must do, how it is built, why it is built that way, and the evidence that it works. It is written for a reviewer who wants to judge the design and for a developer who will maintain it.

1.2 Scope

In scope: product and supplier master data, purchase orders with an approval threshold, goods receipts (full and partial), the stock ledger and stock card, stock adjustments, and the integration events sent to Accounting. Out of scope: the general ledger itself (see KERP-ACC-TR-001), login (done by ERPHUB), multi-warehouse stock, sales and issues to customers, multi-currency.

1.3 Intended audience

- Interviewers and reviewers evaluating backend design and business-logic implementation
- Developers extending the module (for example adding vendor invoices or issues)

1.4 Definitions and acronyms

Term	Meaning
PO	Purchase order: a request to a supplier for goods at agreed unit costs
GR	Goods receipt: a record that goods arrived against a PO; a PO may have several
Stock movement	One append-only ledger row that changes on-hand quantity (RECEIPT, ISSUE, ADJUSTMENT)
Stock card	The movement history of one product, with running balance
Satang	1/100 of a Thai baht; all money is stored as integer satang
Outbox	A table written in the same transaction as a business change, holding events to deliver later
GR/IR (2150)	Goods received / invoice received clearing account in Accounting
ERPHUB	The K-ERP gateway: login, JWT cookie and reverse proxy

1.5 Normative references

- ISO/IEC/IEEE 29148 (requirements), ISO/IEC/IEEE 42010 (architecture description), ISO/IEC/IEEE 29119-3 (test documentation), ISO/IEC 25010 (quality model). They are used as structure, not as a certification claim.

1.6 Document overview

Chapter 2 gives the system context, chapter 3 the requirements, chapter 4 the architecture, chapter 5 the detailed design, chapter 6 the tests and their results,

chapter 7 a quality evaluation, chapter 8 traceability, and chapter 9 limitations and future work.

2. System overview

2.1 Background

A small IT-equipment shop buys stock from a few suppliers. It needs to know what it has, buy more with proper approval, receive deliveries that often arrive in parts, and have the accounting entry for every receipt appear in the ledger without anyone re-typing it.

2.2 Product perspective

The module is one of two business modules behind ERPHUB. Users reach it at `/erp/stock/...` after logging in once at the hub; the hub forwards each request with a signed JWT. The module has its own PostgreSQL database and never reads Accounting's database: it sends events to Accounting through a transactional outbox.

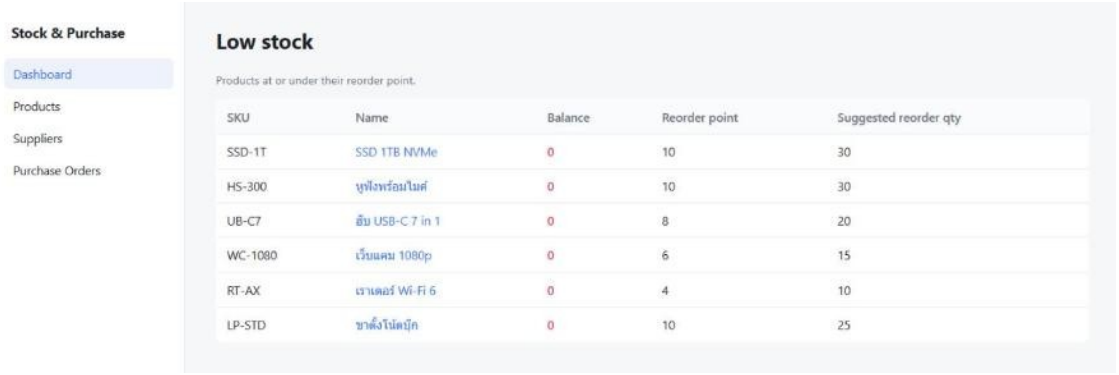


Figure 2.1 Stock dashboard (live demo)

2.3 Product functions

- Maintain products (SKU, unit, standard cost, reorder point/quantity) and suppliers
- Create purchase orders, submit them, approve those over the threshold, send, cancel
- Receive goods against a sent PO, fully or in several partial receipts
- Keep an append-only stock ledger; show live balances, low-stock list and stock card
- Post manual stock adjustments (e.g. after a stock count)
- Emit GOODS_RECEIVED and STOCK_ADJUSTED events to Accounting exactly once

2.4 User classes

Role (JWT `roles`)	Can do in this module
Purchasing	Products, suppliers, create/submit/send/cancel POs
Manager	Approve POs over the threshold; cancel
Warehouse Staff	Receive goods, stock adjustments
Admin	Everything, plus the outbox operations view and retry
Accountant	Read-only here (works in Accounting)

All authenticated users with `stock` in `allowedModules` can read master data, POs, balances and the stock card.

2.5 Operating environment

Part	Technology	Hosting (demo)
API	Node.js 22, Express 4, Prisma 6, Zod	Render Web Service (free), Singapore
Database	PostgreSQL	Neon (free), ap-southeast-1
UI	React + Vite SPA, served by the API at /erp/stock/	same service
Gateway	ERPHUB (Next.js)	Vercel (Hobby)

2.6 Design constraints

- Zero hosting cost: free tiers sleep when idle (≈30 s first request) and have monthly limits
- No login in the module: only verify the hub's HS256 JWT
- No message broker: cross-module delivery must work with PostgreSQL alone
- Money in integer satang; PO totals capped at 20,000,000 THB to stay inside a 32-bit column

2.7 Assumptions and dependencies

- ERPHUB and the module share `JWT_SECRET`; Stock and Accounting share `ACCOUNTING_SERVICE_KEY`
- One API instance (the outbox worker and in-process guard assume it)
- Accounting accepts the posting API described in KERP-ACC-TR-001; until it is configured, events wait as PENDING

3. Software requirements specification

3.1 Functional requirements

ID	Requirement
FR-01	The system shall store products with a unique SKU, unit, standard cost (satang), reorder point and reorder quantity.
FR-02	The system shall store suppliers with contact information and lead time.
FR-03	The system shall create a PO in DRAFT with at least one line and give it a gap-free number PO-YYYY-NNNNN (five digits), the year in Bangkok time.
FR-04	Submitting a PO whose total is above the approval threshold (default 50,000 THB) shall move it to PENDING_APPROVAL; otherwise to APPROVED.
FR-05	Only Manager or Admin shall approve a PENDING_APPROVAL PO.
FR-06	A PO shall follow only the allowed status transitions (§4.7); any other change shall be refused with 409.
FR-07	Goods shall be received only against a SENT or PARTIALLY_RECEIVED PO, and never more than ordered per line.
FR-08	A PO shall become COMPLETED when every line is fully received, otherwise PARTIALLY_RECEIVED.
FR-09	Every change to on-hand quantity shall be recorded as an append-only stock movement with balance before and after.
FR-10	No movement shall take a product's balance below zero (409).
FR-11	The system shall show live balances, a low-stock list (balance ≤ reorder point) and a stock card per product.
FR-12	Each goods receipt and stock adjustment with a non-zero value shall produce exactly one event for Accounting, written in the same transaction.
FR-13	Failed events shall be visible to Admin and retryable after the cause is fixed.

3.2 Business rules

ID	Rule
BR-01	Approval threshold is configurable (`APPROVAL_THRESHOLD_SATANG`, default 5,000,000 satang).
BR-02	Receipt value = Σ qtyReceived × PO line unitCost; Accounting books Dr 1300 Inventory / Cr 2150 GR/IR.
BR-03	Adjustment value = |quantity| × product standard cost; increases Dr 1300 / Cr 5200, decreases Dr 5200 / Cr 1300 (5200 = stock adjustment loss; booked by Accounting).
BR-04	Stock movements are never updated or deleted; corrections are new movements.

ID	Rule
BR-05	A PO cannot be cancelled once receiving has started.

3.3 Quality requirements (per ISO/IEC 25010)

ID	Characteristic	Requirement
QR-01	Functional correctness	Concurrent requests shall never produce a negative or inconsistent stock balance or over-receipt.
QR-02	Reliability (recoverability)	An Accounting outage shall not lose or duplicate postings; delivery resumes on its own.
QR-03	Security	Every API route except /health requires a valid hub JWT with module `stock`; state changes check the role.
QR-04	Maintainability	Modules do not touch each other's tables; business rules have unit or integration tests.
QR-05	Performance	Under 50 concurrent users on one product the API stays correct and p95 latency is recorded (§6.3).

3.4 External interface requirements

3.4.1 User interface

A React SPA served at `/erp/stock/`: dashboard, products, suppliers, purchase orders (list, detail with actions by status, create), goods receipt, stock card, adjustments. Responsive below 760 px.

3.4.2 Software interfaces

- ERPHUB: forwards `Authorization: Bearer <jwt>` with `{ sub, username, roles, allowedModules }`
- Accounting: `POST /api/postings` with header `X-Service-Key`, body `{ topic, payload }`; idempotent per source document

3.4.3 Communication interfaces

HTTPS JSON. Amounts in events are strings of satang. Health: `GET /health` (process) and `GET /health?deep=1` (also the database, 503 when down).

4. Architecture description

4.1 Stakeholders and concerns

Stakeholder	Concern
Purchasing / Manager / Warehouse	Correct POs, approvals and stock figures
Accountant	Every receipt reaches the ledger once, with the right value
Developer	Clear module boundaries, testable rules, safe concurrency
Operator (owner)	Runs at zero cost; failures are visible and recoverable

4.2 Viewpoints

Context, logical (modules), information (data), interaction (outbox), lifecycle (purchase order), deployment and security, following ISO/IEC/IEEE 42010.

4.3 Context view

Browser → ERPHUB (Vercel) → Stock API (Render) → Stock database (Neon). Stock → Accounting API (Render) via the outbox worker. A nightly GitHub Actions job resets the demo data and reconciles stock value with GL account 1300.

4.4 Logical (module) view

Module	Responsibility
auth	Verify the hub JWT; `requireRole` guard. Never issues tokens
product, supplier	Master data
purchase-order	Creation, numbering, status machine, approval threshold
goods-receipt	Receiving against a PO; calls inventory, never writes stock itself
inventory	`postStockMovement` (the only way stock changes), balances, low stock, adjustments
reporting	Stock card
accounting-integration	Pure functions that build Accounting events
shared/outbox	Enqueue in the caller's transaction; worker, backoff, operations view

Rule: a module never queries another module's tables; it calls that module's service.

4.5 Information view

Table	Purpose	Key constraints
products	SKU master	sku unique; cost Int satang

Table	Purpose	Key constraints
suppliers	Supplier master	—
purchase_orders	PO header	poNumber unique; status enum; totalAmount Int satang
purchase_order_lines	Ordered vs received quantity	cascade delete with PO
goods_receipts / goods_receipt_lines	Each delivery against a PO	line refers to a PO line
stock_movements	Append-only ledger	index (productId, createdAt); never updated
document_counters	Gap-free numbering	key = PO-YYYY
outbox_messages	Events to deliver	dedupeKey unique; index (status, destination, nextAt)

4.6 Interaction view: transactional outbox

1. Warehouse posts `POST /api/goods-receipts`.
2. In one transaction: lock the PO row, validate lines, create the receipt, increment qtyReceived, post a stock movement per line (each locks its product row), enqueue GOODS_RECEIVED with dedupe key `accounting:GOODS_RECEIPT:<receiptId>`, set the PO status.
3. Commit. If anything fails, nothing is saved, including the event.
4. Every 5 s the worker sends due PENDING events to Accounting in creation order.
5. 2xx → SENT. 408/429/5xx/network → retry after 5 s, 10 s, 20 s … (max 15 min), up to 8 attempts. Other 4xx → FAILED for a person to fix and retry.
6. Accounting returns the existing entry if the same source document arrives again, so a retry after a lost response cannot double-post.

4.7 Lifecycle view: purchase order

From	Allowed next status	Trigger (role)
DRAFT	PENDING_APPROVAL, APPROVED, CANCELLED	submit (Purchasing) — above/at-or-below threshold; cancel
PENDING_APPROVAL	APPROVED, CANCELLED	approve (Manager); cancel
APPROVED	SENT, CANCELLED	send (Purchasing); cancel
SENT	PARTIALLY_RECEIVED, COMPLETED, CANCELLED	goods receipt (Warehouse); cancel
PARTIALLY_RECEIVED	COMPLETED	goods receipt
COMPLETED, CANCELLED	—	final

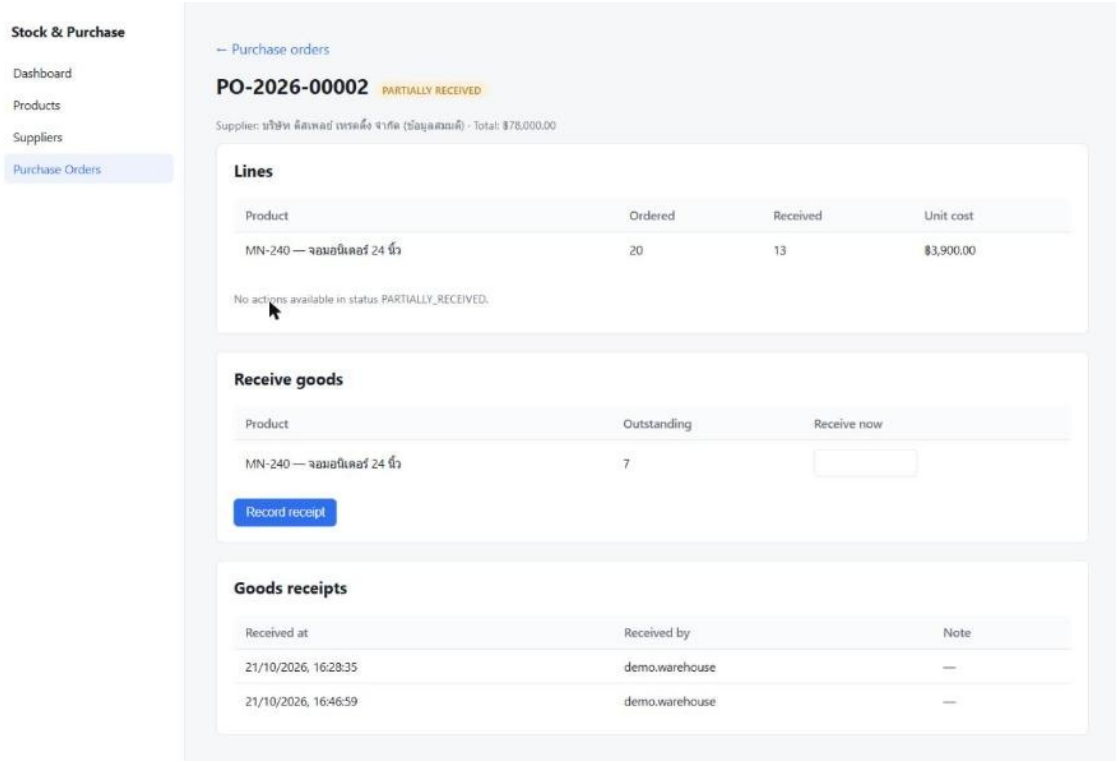


Figure 4.1 A partially received purchase order

4.8 Deployment view

One Render web service builds the API and the sibling frontend (`rm -rf frontend && git clone … && npm ci && npm run build`), runs `prisma migrate deploy` before start, and serves the SPA. The database is a Neon project in Singapore. Configuration is environment variables only (§5.6).

4.9 Security view

- The module is reached only through the hub; tokens live in an httpOnly cookie at the hub and arrive here as a Bearer header
- `requireAuth` rejects missing/invalid tokens (401) and tokens without module `stock` (403); `requireRole` guards each state change (403)
- The UI does not hide buttons by role; the backend is the authority (ADR-0006)
- The service key to Accounting is only in environment variables and never logged
- Input validated with Zod at every route; Prisma parameterises all queries, including the raw `FOR UPDATE` statements

4.10 Architecture decision records

ID	Decision	Rationale	Consequence
ADR-S1	Append-only stock ledger; balance = sum of movements	Audit trail, no lost updates, stock card is a query	Summing slows with history; snapshot table later

ID	Decision	Rationale	Consequence
ADR-S2	Row lock (`SELECT … FOR UPDATE`) on the product before each movement and on the PO before a receipt	Serialises the read-check-write that would otherwise race	Same-product requests queue; tested in §6
ADR-S3	Compare-and-set PO status updates (`updateMany … where status = expected`)	Two concurrent actions on one PO must not both win	The loser gets 409 and reloads
ADR-S4	Transactional outbox, not inline HTTP	Accounting may be down; inline calls lose or duplicate	Ledger trails stock by seconds
ADR-S5	Counter row for PO numbers (`INSERT … ON CONFLICT DO UPDATE … RETURNING`)	Sequences skip numbers on rollback	Creates within one year serialise briefly
ADR-S6	Integer satang everywhere; event amounts as bigint strings	Floats round; qty × cost can exceed 32 bits	Clients parse strings

The system-wide decisions are also recorded as ADR-0001 … ADR-0007 in the K-ERP repository (docs/adr).

5. Detailed design

5.1 Posting a stock movement

`postStockMovement(tx, input)` is the only code path that changes stock. It must run inside the caller's transaction:

1. `SELECT id FROM products WHERE id = $1 FOR UPDATE` — wait for any other transaction moving the same product; 404 if missing.
2. balanceBefore = SUM(quantity) for the product (READ COMMITTED takes a fresh snapshot per statement, so it sees the previous holder's committed row).
3. balanceAfter = balanceBefore + quantity; refuse with 409 if negative.
4. Insert the movement with both balances, type, reference and user.

5.2 Receiving goods

1. Lock the PO row (`FOR UPDATE`), then load it with its lines; 404 / 409 if not SENT or PARTIALLY_RECEIVED.
2. For each line: it must belong to the PO, qty > 0, and received + qty ≤ ordered (409 otherwise).
3. Create the receipt and its lines; increment each PO line's qtyReceived; post a RECEIPT movement per line.
4. Build GOODS_RECEIVED (amount = Σ qty × unitCost as bigint) and enqueue it.
5. Set the PO to COMPLETED if every line is full, else PARTIALLY_RECEIVED.

5.3 Purchase order numbering and status changes

PO numbers come from `document_counters` with key `PO-<Bangkok year>`, incremented by an upsert that takes a row lock inside the creating transaction, so concurrent creates get consecutive numbers with no gaps or duplicates. Status changes check the transition table, then write with `updateMany({ where: { id, status: expected } })`; zero rows updated means another request changed the PO first, answered with 409.

5.4 Accounting events

Topic	Dedupe key	Payload (version 1)
GOODS_RECEIVED	accounting:GOODS_RECEIPT:<receiptId>	occurredAt, goodsReceiptId, poId, poNumber, amountSatang
STOCK_ADJUSTED	accounting:STOCK_ADJUSTMENT:<movementId>	occurredAt, movementId, productId, sku, direction, quantity, reason, amountSatang
OPENING_BALANCE	accounting:OPENING_BALANCE	asOf, amountSatang — one-off cut-over of existing stock value (Dr 1300 / Cr 3200)

Zero-value movements produce no event (Accounting rejects zero amounts).

5.5 Error model

Status	When
400	Zod validation failure, no lines, non-positive quantity, line not on this PO, total over 20,000,000 THB

Status	When
401 / 403	Missing or invalid token / module or role not allowed
404	Unknown id (including Prisma P2025)
409	Illegal status transition, over-receipt, negative stock, PO changed concurrently
500	Anything else (logged; body never contains internals)

5.6 Configuration

Variable	Meaning
DATABASE_URL	PostgreSQL connection string
JWT_SECRET	Same value as ERPHUB
APPROVAL_THRESHOLD_SATANG	PO approval threshold (default 5,000,000)
ACCOUNTING_URL, ACCOUNTING_SERVICE_KEY	Outbox destination; unset = events wait
OUTBOX_WORKER	`off` to disable the worker in this process
FRONTEND_DIST, PORT	SPA build location; listen port

6. Test documentation

6.1 Test plan

Level	Tool	Environment	What it proves
Unit	Vitest	No database	PO state machine, event building, outbox delivery/backoff rules
Integration	Vitest + Prisma	PostgreSQL 16 (GitHub Actions service container)	Row locks, compare-and-set, gap-free numbering, over-receipt
Negative control	Same integration tests	Throwaway branch without `FOR UPDATE`	That the tests fail when the locks are removed
Load	k6	API + PostgreSQL 16 on a GitHub-hosted runner	Correctness and latency with 50 concurrent users on one product
End-to-end	Node script across modules	Both modules + Accounting	Outbox delivery, outage recovery, reconciliation (KERP-ACC-TR-001 §6)
Production smoke	Manual via the hub	Live demo	PO → approve → receive twice → two journal entries

Load tests never target the free demo, so they do not consume its monthly quota.

6.2 Test design

Test case	Setup	Action	Expected
TC-01 Concurrent issues	Product with 30 in stock	50 concurrent adjustments of −1	30 succeed, 20 × 409, balance 0, balanceBefore runs 30…1 with no repeats
TC-02 Concurrent receipts	SENT PO, one line of 10	5 concurrent receipts of 3	3 succeed, 2 × 409, qtyReceived 9, stock 9, PARTIALLY_RECEIVED
TC-03 Concurrent status changes	APPROVED PO	3 sends + 3 cancels at once	Exactly 1 succeeds, 5 × 409, final status = winner's
TC-04 Concurrent PO numbers	Empty counter	20 concurrent creates	Numbers 1…20, no duplicates
TC-05 Oversized PO	—	Create a PO over 20,000,000 THB	400
TC-06 Load on one product	Seeded product	k6: 50 VUs × 30 s of +1 adjustments	< 1% errors; ledger sum = latest balance; no duplicate balanceBefore

6.3 Test completion report

Run on 21 October 2026. CI run: GitHub Actions CI #2 on commit c54d9a5 (github.com/kingly5556/Stock-Purchase-Backend/actions/runs/39626441989).

Suite	Result
Unit: events.test.ts (event building)	8 / 8 pass
Unit: delivery.test.ts (outbox retry rules)	12 / 12 pass
Unit: po-status.test.ts (state machine)	6 / 6 pass
Integration: stock-movement.integration.test.ts (TC-01…TC-03)	3 / 3 pass (656 ms)
Integration: po-number.integration.test.ts (TC-04, TC-05)	2 / 2 pass
Total	31 / 31 pass
Cross-module E2E (KERP-ACC-TR-001 §6)	23 / 23 pass (15 Oct 2026)
Production smoke and outage test (KERP-DEP-RPT-002)	pass (21 Oct 2026)

Negative control: the same tests without the locks

Branch `experiment/no-row-lock` removes both `FOR UPDATE` statements and nothing else. Run: CI #3 on commit a337a62 (actions/runs/39627358290).

Test case	With locks (main)	Without locks (experiment)
TC-01 50 issues vs 30 in stock	30 succeed, 20 × 409, balance 0	FAIL: 32 succeed — 2 units oversold
TC-02 5 receipts of 3 vs 10 ordered	3 succeed, 2 × 409, 9 received	FAIL: all 5 succeed — 15 received against 10 ordered
TC-03 3 sends + 3 cancels	1 succeeds, 5 × 409	pass (compare-and-set is not removed on this branch)

Load test (k6)

Environment: GitHub-hosted ubuntu-latest runner; API (Node 22) and PostgreSQL 16 service container on the same machine; outbox worker off; 50 virtual users posting +1 adjustments to the same product for 30 s. Run: Load test #1 (actions/runs/39626932878).

Metric	Value
Requests	10,275 (340.9 per second)
Failed requests	0.00%
Latency average / median	146.2 ms / 138.7 ms
Latency p90 / p95	156.3 ms / 167.9 ms
Latency max	1,005.3 ms

Metric	Value
Movements written / sum of quantities / latest balance	10,275 / 10,275 / 10,275
Duplicate balanceBefore values	0
Rows where balanceAfter ≠ balanceBefore + quantity	0

6.4 Incident (anomaly) report

ID	Found by	Description	Resolution
INC-01	Code review while writing this report	Goods receipt read the PO without a lock: two concurrent receipts on one line could both pass the ‘not more than ordered’ check and over-receive.	Lock the PO row first (commit ecf4412); TC-02.
INC-02	Code review	PO status changes read then wrote unconditionally: e.g. a cancel could overwrite a receipt that committed in between.	Compare-and-set updates (commit c54d9a5); TC-03.
INC-03	Production (deployment)	Seeding from GitHub Actions (US) to Neon (Singapore) hit Prisma's 5 s interactive-transaction timeout (P2028).	transactionOptions maxWait 10 s / timeout 30 s (commit db3a538).
INC-04	Production (deployment)	Render's build cache kept a stale `frontend/` folder, so `git clone` failed.	Build command starts with `rm -rf frontend`.
INC-05	Negative-control run	The two integration test files share one test database and ran in parallel; one file's cleanup could delete purchase orders the other was using. It passed on main only by timing.	Run test files one at a time and delete receipts first in the cleanup (commit 1834ea5).

7. Quality evaluation (ISO/IEC 25010)

Characteristic	Evidence	Assessment
Functional suitability	FR-01…FR-13 implemented; traceability in chapter 8	Good
Reliability	Outbox + idempotency; production outage test posted once after resume; nightly reconciliation	Good
Performance efficiency	k6 result §6.3; same-product requests serialise by design	Adequate for a small business
Security	JWT verify-only, role guard on every state change, Zod validation, secrets only in env	Good; no rate limiting yet
Maintainability	Module boundaries, pure event builders, state table, tests for every concurrency rule	Good
Usability	SPA with status-driven actions, mobile layout; buttons not hidden by role	Adequate

7.1 Findings and recommendations

- Two real race conditions were found by reviewing the code against the concurrency rules and fixed with tests (INC-01, INC-02). Every new multi-step write should get the same review.
- Balance = SUM over the ledger is correct but grows with history; add a per-product snapshot when movements reach the hundreds of thousands.
- Add rate limiting at the hub and an audit view of who approved what.

8. Requirements traceability matrix

Requirement	Implemented in	Verified by
FR-01, FR-02	product/, supplier/ routes	Production smoke flow
FR-03	purchase-order.service generatePoNumber	TC-04
FR-04, FR-05	submitPurchaseOrder, approve route (Manager/Admin)	Role matrix 80/80 (overview README), smoke flow
FR-06	po-status.ts, updateIfStatus	po-status.test.ts, TC-03
FR-07, FR-08	goods-receipt.service	TC-02, smoke flow
FR-09, FR-10	inventory.service postStockMovement	TC-01, TC-06
FR-11	inventory routes, reporting/stock-card	Smoke flow
FR-12	accounting-integration/events, shared/outbox	events.test.ts, delivery.test.ts, E2E 23/23, production outage test
FR-13	outbox.routes (Admin)	E2E permanent-failure case
QR-01	Row locks, compare-and-set	TC-01…TC-03, negative control, TC-06

9. Limitations, risks and future work

9.1 Intentional limitations

- One warehouse; no sales/issues to customers yet (the ledger already supports ISSUE)
- Standard cost per product for adjustments; no moving-average or FIFO valuation
- One API instance; the outbox worker would need `FOR UPDATE SKIP LOCKED` to scale out

9.2 Risks

Risk	Mitigation
Free hosting sleeps or runs out of hours	Hub shows a waking page; quota check scheduled; paid tier is a config change
Accounting misconfigured (4xx)	Events go FAILED, visible to Admin, retry after the fix
Hot product under heavy load	Requests queue on the row lock; measured in §6.3

9.3 Future work

- Vendor invoices (Dr 2150 / Cr 2100) and VAT on purchases
- Issues to sales orders; reorder suggestions that draft POs
- Per-product balance snapshot; OpenAPI document

10. Conclusion

The module covers the buying-and-stocking process end to end and hands every receipt to the general ledger exactly once. Its main risk, concurrent requests corrupting stock or purchase orders, is handled by row locks and compare-and-set updates, each proven by a test that passes with the protection and fails without it. It runs in production on free hosting with CI on every push.

Appendix A. API reference

All `/api/*` routes need a hub JWT with module `stock`; “any” means any such user.

Method	Path	Roles	Purpose
GET	/health[?deep=1]	none	Process / database health
GET	/api/products, /api/products/:id	any	List / read products
POST, PATCH	/api/products[/:id]	Admin, Purchasing	Create / update product
GET	/api/suppliers, /api/suppliers/:id	any	List / read suppliers
POST, PATCH	/api/suppliers[/:id]	Admin, Purchasing	Create / update supplier
GET	/api/inventory/balance/:productId	any	Live balance
GET	/api/inventory/low-stock	any	Products at or below reorder point
POST	/api/inventory/adjustments	Admin, Warehouse Staff	Stock adjustment
GET	/api/purchase-orders, /:id	any	List / read POs
POST	/api/purchase-orders	Admin, Purchasing	Create DRAFT PO
POST	/api/purchase-orders/:id/submit	Admin, Purchasing	Submit
POST	/api/purchase-orders/:id/approve	Admin, Manager	Approve
POST	/api/purchase-orders/:id/send	Admin, Purchasing	Send to supplier
POST	/api/purchase-orders/:id/cancel	Admin, Purchasing, Manager	Cancel
POST	/api/goods-receipts	Admin, Warehouse Staff	Receive goods
GET	/api/reports/stock-card/:productId	any	Stock card
GET	/api/outbox	Admin	Integration events by status
POST	/api/outbox/:id/retry	Admin	Retry a FAILED event

Appendix B. Data dictionary

stock_movements

Column	Type	Notes
id	text (cuid)	PK
productId	text	FK products
type	enum	RECEIPT | ISSUE | ADJUSTMENT
quantity	int	Signed
balanceBefore, balanceAfter	int	balanceAfter = balanceBefore + quantity
refType, refId	text	e.g. GOODS_RECEIPT / receipt id
createdBy	text	JWT sub
createdAt	timestamp	Indexed with productId

purchase_orders / purchase_order_lines

Column	Type	Notes
poNumber	text	Unique, PO-YYYY-NNNNN
status	enum	See §4.7
totalAmount	int	Satang, ≤ 2,000,000,000
createdBy, approvedBy, approvedAt	text / timestamp	Audit of who did what
qtyOrdered, qtyReceived, unitCost	int	Per line; qtyReceived ≤ qtyOrdered

outbox_messages

Column	Type	Notes
topic, destination	text	e.g. GOODS_RECEIVED → accounting
dedupeKey	text	Unique
payload	jsonb	{ version, occurredAt, … }
status	enum	PENDING | SENT | FAILED
attempts, lastError, nextAt, sentAt	int / text / timestamp	Retry bookkeeping